Projeto: Sistema de Gestão "Auto Peças do Baiano"

1. Contextualização do Problema

No mercado atual, a informatização é fundamental para a sobrevivência e crescimento de qualquer negócio. O Sr. Baiano, proprietário da tradicional **"Auto Peças do Baiano"**, percebeu que o controle de seu estoque no caderno e em planilhas soltas estava gerando prejuízos. Peças sumiam, preços ficavam desatualizados e o atendimento ao cliente no balcão estava cada vez mais lento.

Para modernizar a loja, ele encomendou o desenvolvimento de um sistema web focado em gerenciar o seu catálogo de produtos. A aplicação precisa ser segura, permitindo o acesso apenas de funcionários autorizados, e deve oferecer operações completas de cadastro, leitura, atualização e exclusão (CRUD) das peças disponíveis no estoque.

2. Objetivos de Aprendizagem do Laboratório

Este projeto prático não é apenas sobre entregar um software funcionando, mas sim sobre compreender as engrenagens por trás da web. Antes de migrarmos para ferramentas de mercado automatizadas, é essencial dominarmos a base. Ao longo das próximas quatro aulas, o objetivo é construir essa aplicação do zero, aplicando os seguintes conceitos:

- **Arquitetura MVC Puro (Model, View, Controller):** Compreender a separação de responsabilidades na prática, sem depender de frameworks externos. Vocês entenderão como a requisição sai da tela, passa pelo controlador, busca a regra de negócio no modelo e retorna para o usuário.
- **Programação Orientada a Objetos com PHP:** Estruturar classes e métodos para lidar com a lógica de negócio e a conexão com o banco de dados.
- **Persistência e Segurança de Dados:** Criar a estrutura no MySQL, utilizar a interface PDO para prevenir Injeção de SQL e aplicar o conceito de hash para armazenamento seguro de senhas de acesso.

3. Escopo e Metodologia

Para garantir o foco absoluto na lógica de backend, no fluxo de dados e na arquitetura do sistema, **não utilizaremos nenhuma estilização visual (CSS, Bootstrap ou templates externos)**. As interfaces (Views) serão construídas apenas com HTML puro.

O desenvolvimento será feito em servidor local (localhost) e dividido em etapas incrementais. Cada aula trará um novo desafio, começando pela infraestrutura de banco de dados, passando pelo sistema de login, até chegarmos à gestão completa dos produtos da Auto Peças do Baiano. Mãos à obra!

---

1. Estrutura de Diretórios

Criem uma pasta chamada `autopecas` no diretório raiz do servidor local (como o `htdocs` do XAMPP). A organização será estritamente baseada no padrão MVC:

```
autopecas/
│
├── config/
```

```
│   └── database.php        (Responsável pela conexão com o banco)
│
├── controllers/
│   ├── AuthController.php  (Gerencia login/logout)
│   └── ProdutoController.php (Gerencia o CRUD das peças)
│
├── models/
│   ├── Usuario.php         (Regras de negócio do usuário)
│   └── Produto.php         (Regras de negócio das peças)
│
├── views/
│   ├── login.php           (Formulário de acesso)
│   ├── dashboard.php       (Menu principal após logar)
│   ├── produto_form.php    (Formulário de cadastro/edição)
│   └── produto_list.php    (Tabela listando as peças)
│
└── index.php               (Front Controller - roteamento básico da aplicação)
```

---

2. Script do Banco de Dados (MySQL)

Este script deve ser executado no `WorkBench`. Ele cria o banco, as tabelas e já insere um usuário padrão para que eles possam testar o login.

```
-- Criar o banco de dados
CREATE DATABASE IF NOT EXISTS autopecas_db;
USE autopecas_db;

-- Tabela de Usuários (Login)
CREATE TABLE IF NOT EXISTS usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL
);

-- Inserir um usuário administrador padrão (Senha: 123456)
-- Nota: Em produção, usaríamos password_hash() no PHP, mas para simplificar
-- o primeiro teste de laboratório, inserimos a hash MD5 de "123456" ou plain text.
-- Aqui usaremos uma hash simples MD5 para a senha '123456'.
INSERT INTO usuarios (usuario, senha) VALUES ('admin', MD5('123456'));

-- Tabela de Produtos (Autopeças)
CREATE TABLE IF NOT EXISTS produtos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    codigo_peca VARCHAR(50) NOT NULL UNIQUE,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10, 2) NOT NULL,
    quantidade_estoque INT NOT NULL DEFAULT 0,
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Roteiro Passo a Passo (Distribuição das 4 Aulas)

Aula 1: Infraestrutura, Banco de Dados e Conexão

- **Objetivo:** Preparar o ambiente, rodar o script MySQL, criar a estrutura de pastas e estabelecer a conexão com o banco usando **PDO** (PHP Data Objects).
- **Ação do Aluno:** Criar o arquivo `config/database.php`.

```
<?php
// Arquivo: config/database.php
class Database {
    private $host = "localhost";
    private $db_name = "autopecas_db";
    private $username = "root"; // Usuário padrão do XAMPP
    private $password = "";     // Senha padrão do XAMPP (vazia)
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" . $this-
>db_name, $this->username, $this->password);
            $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
            echo "Erro na conexão: " . $exception->getMessage();
        }
        return $this->conn;
    }
}
?>
```

Aula 2: Sistema de Autenticação (Login e Logout)

- **Objetivo:** Implementar o padrão MVC para o controle de acesso.
- **Ação do Aluno:** 1. Criar o `models/Usuario.php` (com método para verificar credenciais no banco). 2. Criar a `views/login.php` (um HTML simples com `<form>`, inputs e botão de submit). 3. Criar o `controllers/AuthController.php` para receber o POST, iniciar a `session_start()` e validar o usuário. 4. Criar o `index.php` na raiz, que servirá de "ponto de entrada" para decidir se exibe o login ou a dashboard.

Aula 3: CRUD de Produtos - Parte 1 (Create e Read)

- **Objetivo:** Proteger as páginas exigindo sessão ativa e criar a listagem e cadastro de peças.
- **Ação do Aluno:**
    1. Criar a `views/dashboard.php` contendo links em HTML puro (Ex: "Cadastrar Nova Peça", "Listar Peças", "Sair").
    2. Criar o `models/Produto.php` com métodos `cadastrar()` e `listarTodos()`.

3. Criar a `views/produto_form.php` (para o INSERT) e `views/produto_list.php` (uma tabela HTML `<table>` com os dados do banco).
4. Criar o `controllers/ProdutoController.php` para ligar as Views ao Model.

Aula 4: CRUD de Produtos - Parte 2 (Update e Delete) e Finalização

- **Objetivo:** Fechar o ciclo de vida dos dados e amarrar rotas.
- **Ação do Aluno:**
  1. Adicionar links de "Editar" e "Excluir" na tabela construída na Aula 3, passando o ID via GET (`?acao=editar&id=1`).
  2. Adicionar os métodos `buscarPorId()`, `atualizar()` e `deletar()` no `models/Produto.php`.
  3. Criar as ações correspondentes no `ProdutoController.php`.

Aula 2: Sistema de Autenticação Segura (Login e Logout)

Nesta aula, vocês vão criar a validação de acesso e proteger o sistema usando sessões (`$_SESSION`).

1. O "Truque" da Senha (Script Auxiliar) Como adotamos o `password_hash`, os alunos não podem inserir `123456` direto no banco. Peça para criarem um arquivo temporário na raiz chamado `gerar_senha.php`:

```
<?php
// O aluno roda este arquivo uma vez no localhost para descobrir o hash
echo password_hash('123456', PASSWORD_DEFAULT);
?>
```

Eles devem pegar o resultado impresso na tela e executar um `UPDATE` ou `INSERT` no MySQL para gravar essa string no campo `senha` do usuário `admin`.

2. Model: `models/Usuario.php` Responsável por ir ao banco e usar o `password_verify` nativo do PHP.

```
<?php
require_once '../config/database.php';

class Usuario {
    private $conn;

    public function __construct() {
        $db = new Database();
        $this->conn = $db->getConnection();
    }

    public function autenticar($usuario, $senha_digitada) {
        $query = "SELECT id, senha FROM usuarios WHERE usuario = :usuario";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':usuario', $usuario);
        $stmt->execute();

        if ($stmt->rowCount() > 0) {
            $row = $stmt->fetch(PDO::FETCH_ASSOC);
```

```
            // Valida a senha em texto puro contra o hash do banco
            if (password_verify($senha_digitada, $row['senha'])) {
                return $row['id'];
            }
        }
        return false;
    }
}
?>
```

3. View: `views/login.php` Formulário simples apontando para o Controller de autenticação.

```
<!DOCTYPE html>
<html>
<head><title>Login - Autopeças</title></head>
<body>
    <h2>Acesso ao Sistema</h2>
    <form action="../controllers/AuthController.php?acao=login" method="POST">
        <label>Usuário:</label><br>
        <input type="text" name="usuario" required><br><br>

        <label>Senha:</label><br>
        <input type="password" name="senha" required><br><br>

        <button type="submit">Entrar</button>
    </form>
</body>
</html>
```

4. Controller: `controllers/AuthController.php` Gerencia o fluxo de sessão.

```
<?php
session_start();
require_once '../models/Usuario.php';

$acao = $_GET['acao'] ?? '';

if ($acao == 'login') {
    $usuario_form = $_POST['usuario'];
    $senha_form = $_POST['senha'];

    $usuarioModel = new Usuario();
    $user_id = $usuarioModel->autenticar($usuario_form, $senha_form);

    if ($user_id) {
        $_SESSION['usuario_logado'] = $user_id;
        header("Location: ../controllers/ProdutoController.php?acao=dashboard");
    } else {
        echo "Login ou senha inválidos! <a href='../views/login.php'>Voltar</a>";
    }
} elseif ($acao == 'logout') {
```

```
    session_destroy();
    header("Location: ../views/login.php");
}
?>
```

Aula 3: Dashboard e Listagem (O "R" do CRUD)

Aqui eles começam a lidar com a tabela de produtos, garantindo que as páginas só abram se o usuário estiver logado.

1. Model: `models/Produto.php` **(Parte 1)**

```
<?php
require_once '../config/database.php';

class Produto {
    private $conn;

    public function __construct() {
        $db = new Database();
        $this->conn = $db->getConnection();
    }

    public function listarTodos() {
        $query = "SELECT * FROM produtos ORDER BY id DESC";
        $stmt = $this->conn->prepare($query);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
?>
```

2. View: `views/dashboard.php` O menu principal da aplicação.

```
<!DOCTYPE html>
<html>
<head><title>Dashboard</title></head>
<body>
    <h2>Painel da Autopeças</h2>
    <p><a href="../controllers/ProdutoController.php?acao=listar">Listar
Produtos</a></p>
    <p><a href="../controllers/ProdutoController.php?acao=novo">Cadastrar Novo
Produto</a></p>
    <hr>
    <p><a href="../controllers/AuthController.php?acao=logout">Sair do Sistema</a>
</p>
</body>
</html>
```

3. View: `views/produto_list.php` Apresentação dos dados em tabela HTML pura.

```
<!DOCTYPE html>
<html>
<head><title>Lista de Peças</title></head>
<body>
    <h2>Peças em Estoque</h2>
    <a href="../controllers/ProdutoController.php?acao=dashboard">Voltar ao
Dashboard</a>
    <br><br>
    <table border="1">
        <tr>
            <th>ID</th>
            <th>Código</th>
            <th>Nome</th>
            <th>Preço</th>
            <th>Estoque</th>
            <th>Ações</th>
        </tr>
        <?php foreach ($produtos as $p): ?>
        <tr>
            <td><?= $p['id'] ?></td>
            <td><?= $p['codigo_peca'] ?></td>
            <td><?= $p['nome'] ?></td>
            <td>R$ <?= number_format($p['preco'], 2, ',', '.') ?></td>
            <td>
                <a href="../controllers/ProdutoController.php?acao=editar&id=<?=
$p['id'] ?>">Editar</a> |
                <a href="../controllers/ProdutoController.php?acao=excluir&id=<?=
$p['id'] ?>" onclick="return confirm('Tem certeza?');">Excluir</a>
            </td>
        </tr>
        <?php endforeach; ?>
    </table>
</body>
</html>
```

4. Controller: `controllers/ProdutoController.php` **(Parte 1)** O maestro das rotas de produtos. Deve sempre verificar a sessão primeiro.

```
<?php
session_start();
if (!isset($_SESSION['usuario_logado'])) {
    header("Location: ../views/login.php");
    exit;
}

require_once '../models/Produto.php';
$acao = $_GET['acao'] ?? 'dashboard';
$produtoModel = new Produto();
```

```
if ($acao == 'dashboard') {
    require_once '../views/dashboard.php';
} elseif ($acao == 'listar') {
    $produtos = $produtoModel->listarTodos();
    require_once '../views/produto_list.php';
}
// Os próximos IFs serão feitos na Aula 4
?>
```

Aula 4: Conclusão do CRUD (Create, Update e Delete)

Na última aula, os alunos fecham o ciclo adicionando os métodos de escrita no banco.

1. Completando o Model (`models/Produto.php`) Oriente os alunos a adicionarem estes três métodos na classe `Produto`:

```
    public function cadastrar($codigo, $nome, $descricao, $preco, $estoque) {
        $query = "INSERT INTO produtos (codigo_peca, nome, descricao, preco,
quantidade_estoque) VALUES (:codigo, :nome, :descricao, :preco, :estoque)";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':codigo', $codigo);
        $stmt->bindParam(':nome', $nome);
        $stmt->bindParam(':descricao', $descricao);
        $stmt->bindParam(':preco', $preco);
        $stmt->bindParam(':estoque', $estoque);
        return $stmt->execute();
    }

    public function buscarPorId($id) {
        $query = "SELECT * FROM produtos WHERE id = :id";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':id', $id);
        $stmt->execute();
        return $stmt->fetch(PDO::FETCH_ASSOC);
    }

    public function deletar($id) {
        $query = "DELETE FROM produtos WHERE id = :id";
        $stmt = $this->conn->prepare($query);
        $stmt->bindParam(':id', $id);
        return $stmt->execute();
    }
```

2. View: `views/produto_form.php` Um único formulário que serve tanto para Cadastrar quanto para Editar.

```
<!DOCTYPE html>
<html>
<head><title>Formulário de Peça</title></head>
<body>
```

```
    <h2><?= isset($produto) ? 'Editar Peça' : 'Nova Peça' ?></h2>
    <form action="../controllers/ProdutoController.php?acao=salvar" method="POST">

        <input type="hidden" name="id" value="<?= $produto['id'] ?? '' ?>">

        <label>Código da Peça:</label><br>
        <input type="text" name="codigo_peca" value="<?= $produto['codigo_peca'] ??
'' ?>" required><br><br>

        <label>Nome:</label><br>
        <input type="text" name="nome" value="<?= $produto['nome'] ?? '' ?>"
required><br><br>

        <label>Descrição:</label><br>
        <textarea name="descricao"><?= $produto['descricao'] ?? '' ?></textarea><br>
<br>

        <label>Preço:</label><br>
        <input type="number" step="0.01" name="preco" value="<?= $produto['preco']
?? '' ?>" required><br><br>

        <label>Quantidade em Estoque:</label><br>
        <input type="number" name="quantidade_estoque" value="<?=
$produto['quantidade_estoque'] ?? '' ?>" required><br><br>

        <button type="submit">Salvar Dados</button>
    </form>
    <br>
    <a href="../controllers/ProdutoController.php?acao=listar">Cancelar</a>
</body>
</html>
```

3. Completando o Controller (`controllers/ProdutoController.php`) Adicione estas condições ao final do arquivo:

```
// ... continuação do ProdutoController.php

elseif ($acao == 'novo') {
    require_once '../views/produto_form.php';
} elseif ($acao == 'editar') {
    $id = $_GET['id'];
    $produto = $produtoModel->buscarPorId($id);
    require_once '../views/produto_form.php';
} elseif ($acao == 'salvar') {
    $id = $_POST['id'];
    $codigo = $_POST['codigo_peca'];
    $nome = $_POST['nome'];
    $descricao = $_POST['descricao'];
    $preco = $_POST['preco'];
    $estoque = $_POST['quantidade_estoque'];

    if (empty($id)) {
```

```
        // Se não tem ID, é cadastro novo
        $produtoModel->cadastrar($codigo, $nome, $descricao, $preco, $estoque);
    } else {
        // Se tem ID, é edição (O método atualizar não foi colocado no model acima
para economizar espaço,
        // mas seria um UPDATE simples seguindo a lógica do cadastrar)
        // $produtoModel->atualizar($id, $codigo, $nome, $descricao, $preco,
$estoque);
    }
    header("Location: ../controllers/ProdutoController.php?acao=listar");
} elseif ($acao == 'excluir') {
    $id = $_GET['id'];
    $produtoModel->deletar($id);
    header("Location: ../controllers/ProdutoController.php?acao=listar");
}
```

Excelente adaptação, professor! Usar o servidor embutido do PHP (`php -S`) é uma prática fantástica para os alunos, pois desvincula o desenvolvimento da dependência de um diretório específico como o `htdocs` e aproxima a experiência do que eles verão ao usar frameworks no futuro.

Como o PHP vai rodar pelo terminal, o MySQL ainda precisará estar ativo (seja pelo XAMPP, outro serviço ou Docker). Aqui estão as instruções finais reformuladas para entregar aos alunos:

Instruções para Rodar a Aplicação (Servidor Embutido PHP)

Passo 1: Preparar o Banco de Dados (MySQL)

1. Certifique-se de que o serviço do **MySQL** está rodando em sua máquina (você pode iniciá-lo pelo painel do XAMPP ou pelo seu gerenciador de banco de dados preferido).
2. Abra o seu cliente de banco de dados (ex: `http://localhost/phpmyadmin`, MySQL Workbench, DBeaver, etc.).
3. Execute o script SQL fornecido na Aula 1 para criar o banco `autopecas_db` e as tabelas `usuarios` e `produtos`.

Passo 2: Estruturar as Pastas do Projeto

1. Crie uma pasta chamada `autopecas` em qualquer lugar do seu computador (ex: na sua pasta *Documentos* ou *Área de Trabalho*).
2. Salve todos os arquivos criados nas aulas (as pastas `config`, `controllers`, `models`, `views` e o arquivo `index.php`) dentro desta pasta `autopecas`.

Passo 3: Iniciar o Servidor Embutido do PHP

1. Abra o terminal (Prompt de Comando, PowerShell ou o terminal do VS Code).
2. Navegue até a pasta raiz do seu projeto (onde está o `index.php`). Exemplo:

   ```
   cd C:\Usuarios\SeuNome\Documentos\autopecas
   ```

3. Inicie o servidor embutido do PHP apontando para a porta 8000 com o comando:

```
php -S localhost:8000
```

(Atenção: Mantenha esta janela do terminal aberta enquanto estiver desenvolvendo. Se fechá-la, o servidor irá parar).

Passo 4: Configurar a Senha Segura no Banco

Como nossa aplicação usa criptografia `password_hash()`, precisamos gerar a hash da senha padrão (`123456`) antes de fazer o primeiro login.

1. Crie um arquivo temporário chamado `gerar_senha.php` na raiz do seu projeto (junto ao `index.php`) com o seguinte código:

```
<?php echo password_hash('123456', PASSWORD_DEFAULT); ?>
```

2. Abra o navegador e acesse: `http://localhost:8000/gerar_senha.php`
3. Copie o código gerado na tela (a hash da senha).
4. Volte ao seu gerenciador de banco de dados (ex: phpMyAdmin), vá na tabela `usuarios` e edite o usuário `admin`.
5. Cole a hash que você copiou no campo `senha` e salve.
6. *Opcional, mas recomendado:* Exclua o arquivo `gerar_senha.php` do projeto.

Passo 5: Acessar a Aplicação

1. Abra o navegador e acesse a raiz do seu sistema: `http://localhost:8000`
2. O arquivo `index.php` fará o redirecionamento automático para a tela de login.
3. Entre com as credenciais:
   - **Usuário:** admin
   - **Senha:** 123456
4. Teste as funcionalidades de cadastro, listagem, edição e exclusão de peças para validar o funcionamento da arquitetura MVC.